كلية العلوم بتونس
FACULTE DES SCIENCES DE TUNIS

Module DevOps · IGL5 — Groupe 2 · Rapport de projet

Pipeline CI/CD, DevSecOps et GitOps du projet Events

Du `git push` à la supervision sur AWS : un seul pipeline pour cinq features, une infrastructure entièrement décrite en code, un déploiement GitOps sur Kubernetes et une observabilité branchée dès le premier déploiement.

GitHub · GitFlow · Jenkins · Maven · SonarQube · Gitleaks · Trivy · Nexus · Docker · Amazon ECR · Kubernetes EKS · Helm · Argo CD · Terraform · Ansible · Prometheus · Alertmanager · Grafana · OWASP ZAP · Slack

Réalisé par

Roudayna Kouka · Farah Baraket · Ahlem Mastouri · Nour Mustapha · Emna Ouarteni

Formation

Ingénierie du logiciel (IGL5), Faculté des Sciences de Tunis

Date

Octobre 2026

Sommaire

	En bref	3
1	Contexte et objectifs	4
2	Équipe et répartition des features	5
3	Démarche : un pipeline, cinq features	5
4	Vue d'ensemble de l'architecture	7
5	Infrastructure AWS	8
6	Infrastructure as Code : Terraform, Ansible, Argo CD	10
7	Pipeline CI/CD : les étapes en un coup d'œil	13
8	Les étapes du pipeline expliquées une par une	14
	Phase 1 · intégration continue (étapes 0 à 6)	14
	Phase 2 · artefacts et image (étapes 7 à 11)	16
	Phase 3 · déploiement GitOps et vérification (étapes 12 à 16)	17
9	Plateforme Kubernetes	19
10	Observabilité et système de notification	20
11	Sécurité de bout en bout (DevSecOps)	21
12	Justification des choix technologiques	22
13	Mise en œuvre : exécuter le pipeline sur sa feature	23
14	Limites actuelles et propositions d'amélioration	24
15	Conclusion	25
A	Glossaire	26

Comment lire ce rapport

Six schémas structurent le document : la vue d'ensemble (figure 1), l'infrastructure AWS (figure 2), l'Infrastructure as Code (figure 3), les étapes du pipeline (figure 4), la plateforme Kubernetes (figure 5) et la chaîne d'observabilité (figure 6). Le texte qui suit chaque schéma l'explique. La section 8 suit un exemple concret de bout en bout : **Nour Mustapha pousse une modification sur `feature/event`**, ce qui lance le **build n° 12**.

En bref

Ce rapport présente la chaîne DevOps complète de l'application **Events** de la Faculté des Sciences de Tunis. Un `git push` sur une feature déclenche automatiquement la construction, les tests, les contrôles de qualité et de sécurité, la publication des artefacts, le déploiement sur Kubernetes (AWS) et la supervision. Toute l'infrastructure est décrite en code, et chaque déploiement est tracé dans Git.

1
Jenkinsfile pour les 5 features de l'équipe

17
étapes automatisées, du git push à la supervision

6
contrôles de sécurité intégrés au pipeline

0
ressource AWS créée à la main dans la console

Les cinq décisions structurantes

1. **Jenkins pour l'intégration, Argo CD pour le déploiement.** Jenkins construit, teste, contrôle et publie, mais n'a aucun accès au cluster. Il écrit la nouvelle version dans le dépôt plateforme, et Argo CD l'applique depuis l'intérieur du cluster (GitOps).
2. **Un seul pipeline pour cinq features.** Un Jenkinsfile unique et un job *Multibranch* : le responsable, l'environnement, le tag de l'image et le projet SonarQube sont déduits du nom de la branche.
3. **Des contrôles qui bloquent vraiment.** Tests, recherche de secrets, porte qualité SonarQube et tests de fumée arrêtent le pipeline sur toutes les branches. Trivy et OWASP ZAP deviennent bloquants sur `release` et `main`.
4. **Une infrastructure entièrement décrite en code.** Terraform crée, Ansible configure, Argo CD déploie : la plateforme se détruit et se recrée à l'identique.
5. **Une observabilité branchée dès le premier déploiement.** Chaque environnement est découvert par Prometheus, visible dans Grafana, et les incidents arrivent sur Slack.

L'équipe en un coup d'œil

Membre	Branche	Feature	État
Roudayna Kouka	`feature/event-statistics`	Statistiques et tableau de bord FST, intégration DevOps	LIVRÉE
Farah Baraket	`feature/participant`	Participants	EXISTANTE
Ahlem Mastouri	`feature/logistics`	Logistique : recherche, validation, erreurs	À CRÉER
Nour Mustapha	`feature/event`	Événements	EXISTANTE
Emna Ouarteni	`feature/feedback`	Avis et notes des participants	NOUVELLE

1 Contexte et objectifs

1.1 Le projet Events

Events est une application **Spring Boot** de gestion des événements de la faculté : création d'événements, affectation de participants avec leur tâche (organisateur, animateur, serveur, invité), gestion de la logistique (matériel, prix, réservation) et calcul automatique du coût. Elle expose une **API REST**, un **tableau de bord web** aux couleurs de la FST, et s'appuie sur une base **MySQL**. Le code est testé avec JUnit et Mockito, et instrumenté avec Spring Actuator et Micrometer pour la supervision.

1.2 Ce que demande le module

Chaque membre de l'équipe développe sa propre fonctionnalité sur sa branche Git, et **doit exécuter le pipeline DevOps sur sa feature**. Le pipeline doit couvrir toute la chaîne : construction, tests, qualité, sécurité, publication des artefacts, conteneurisation, déploiement et supervision.

> **Notre réponse :** un seul pipeline, écrit une seule fois et versionné avec le code, qui s'adapte automatiquement à la branche qui l'exécute. Chaque membre obtient son build, ses rapports, son image et son propre environnement en ligne, sans dupliquer de configuration.

1.3 Objectifs

Objectif	Comment il est atteint
Automatiser	Un `git push` suffit : du build jusqu'au déploiement, sans aucune action manuelle (sauf l'approbation de la production).
Garantir la qualité	Tests avec et sans Mockito, couverture JaCoCo, analyse SonarQube et porte qualité bloquante.
Sécuriser (DevSecOps)	Six contrôles intégrés : secrets (Gitleaks), dépendances et image (Trivy), code (SonarQube), application en marche (OWASP ZAP), scan ECR.
Isoler chaque feature	Un environnement Kubernetes par feature, créé automatiquement par Argo CD, avec sa propre base de données et son URL.
Tracer	Chaque image porte la feature, l'auteur et le commit. Chaque déploiement est un commit Git.
Reproduire	Toute l'infrastructure AWS est décrite en code (Terraform, Ansible) : détruite et recréée à l'identique en une commande.
Superviser	Prometheus, Alertmanager et Grafana surveillent chaque environnement. L'équipe est prévenue sur Slack.

1.4 Principes directeurs

Shift-left
Les contrôles s'exécutent le plus tôt possible, là où un problème coûte le moins cher à corriger.

Tout est code
Pipeline, infrastructure, configuration des outils, déploiements et alertes sont versionnés dans Git.

GitOps
Git décrit ce qui doit tourner. Argo CD applique et corrige toute dérive du cluster.

Moindre privilège
Rôles IAM au lieu de clés, Jenkins sans droits sur le cluster, pods sans privilèges.

Build once, deploy many
Le JAR testé est celui qui part dans l'image, et l'image testée est celle qui est déployée.

Un pipeline pour tous
Les cinq membres passent exactement par les mêmes étapes et les mêmes contrôles.

2 Équipe et répartition des features

Membre	Branche	Périmètre fonctionnel	API	État
Roudayna Kouka	`feature/event-statistics`	Statistiques par événement et globales (participants par tâche, coût de la logistique réservée), tableau de bord web FST. Également : socle du projet et intégration DevOps.	`/event-stats/*`	LIVRÉE
Farah Baraket	`feature/participant`	CRUD des participants, changement de tâche.	`/participant/*`	EXISTANTE
Ahlem Mastouri	`feature/logistics`	Logistique : le CRUD existe déjà. À compléter par la recherche par mot-clé (méthode `findByDescriptionContaining` déjà prévue), la validation des données (prix ≥ 0, quantité ≥ 1) et une erreur 404 propre.	`/logistics/*`	À CRÉER
Nour Mustapha	`feature/event`	Événements : création, affectation des participants et de la logistique, calcul planifié du coût.	`/event/*`	EXISTANTE
Emna Ouarteni	`feature/feedback`	**Nouvelle feature : avis et notes.** Entité `Feedback` (note de 1 à 5, commentaire) liée à `Event` et `Participant`. Un seul avis par participant, une fois l'événement terminé. Note moyenne par événement.	`/feedback/*`	NOUVELLE

La correspondance entre branche et responsable est centralisée à un seul endroit du pipeline : modifier la répartition revient à changer une ligne.

3 Démarche : un pipeline, cinq features

3.1 Pourquoi un seul pipeline ?

Écrire cinq pipelines, un par membre, serait une erreur classique : cinq copies qui divergent au fil des corrections, et aucune garantie que tout le monde passe par les mêmes contrôles. Nous utilisons un **job Jenkins de type *Multibranch Pipeline*** et un **Jenkinsfile unique** placé à la racine du dépôt. Jenkins découvre automatiquement chaque branche et y exécute ce même fichier. Toutes les différences sont **déduites du nom de la branche** :

Membre	Branche	Environnement	Tag de l'image	Projet SonarQube	URL
Roudayna	`feature/event-statistics`	`events-event-statistics`	`event-statistics-<n>-<sha>`	`fst-events-event-statistics`	`event-statistics.<ip>.sslip.io`
Farah	`feature/participant`	`events-participant`	`participant-<n>-<sha>`	`fst-events-participant`	`participant.<ip>.sslip.io`
Ahlem	`feature/logistics`	`events-logistics`	`logistics-<n>-<sha>`	`fst-events-logistics`	`logistics.<ip>.sslip.io`
Nour	`feature/event`	`events-event`	`event-12-a1b2c3d`	`fst-events-event`	`event.<ip>.sslip.io`
Emna	`feature/feedback`	`events-feedback`	`feedback-<n>-<sha>`	`fst-events-feedback`	`feedback.<ip>.sslip.io`

3.2 GitFlow : le comportement dépend du type de branche

Branche	Ce que fait le pipeline	Environnement	Contrôles de sécurité
`feature/<nom>`	Toutes les étapes sauf l'approbation	`events-<nom>` · test, données de démo	RAPPORT
`Pull Request`	Intégration continue seulement (étapes 0 à 6)	aucun	RAPPORT
`develop`	Toutes les étapes sauf l'approbation	`events-staging` · intégration	RAPPORT
`release/x.y.z`	Toutes les étapes sauf l'approbation	`events-staging` · recette	STRICT
`main`	Toutes les étapes, avec approbation manuelle	`events-prod` · 2 à 5 pods	STRICT

Une feature part de `develop` et y revient par une **Pull Request**, relue par un coéquipier, et fusionnable seulement si le pipeline est vert. Les contrôles de sécurité sont en mode **rapport** sur les features, parce que Spring Boot 2.7.6 a des failles connues qui ne viennent pas du code de l'équipe. Ils deviennent **bloquants** sur `release` et `main`, là où le risque est réel.

3.3 Deux dépôts et le principe GitOps

Dépôt application

`IGL5-G2-EventsProjetcs` : code Spring Boot, tests, `Jenkinsfile`, `Dockerfile`. Modifié par l'équipe via Pull Request.

Dépôt plateforme

`IGL5-G2-Events-Platform` : Terraform, Ansible, chart Helm, manifests Argo CD, et l'**état voulu** de chaque environnement. Modifié par l'équipe et par Jenkins.

GitOps signifie que **Git décrit ce qui doit tourner**, et qu'un agent dans le cluster, Argo CD, met le cluster en conformité en continu. Jenkins ne déploie donc jamais directement : il écrit la nouvelle version dans le dépôt plateforme. Trois avantages en découlent :

- **sécurité** : Jenkins n'a aucun accès au cluster ;
- **traçabilité** : chaque déploiement est un commit ;
- **fiabilité** : toute modification manuelle du cluster est automatiquement annulée.

4 Vue d'ensemble de l'architecture

Le schéma ci-dessous montre le parcours d'une modification, du poste du développeur jusqu'au cluster Kubernetes, et le rôle de chaque outil. Les couleurs correspondent aux familles d'outils, et reviennent dans tous les schémas du rapport.

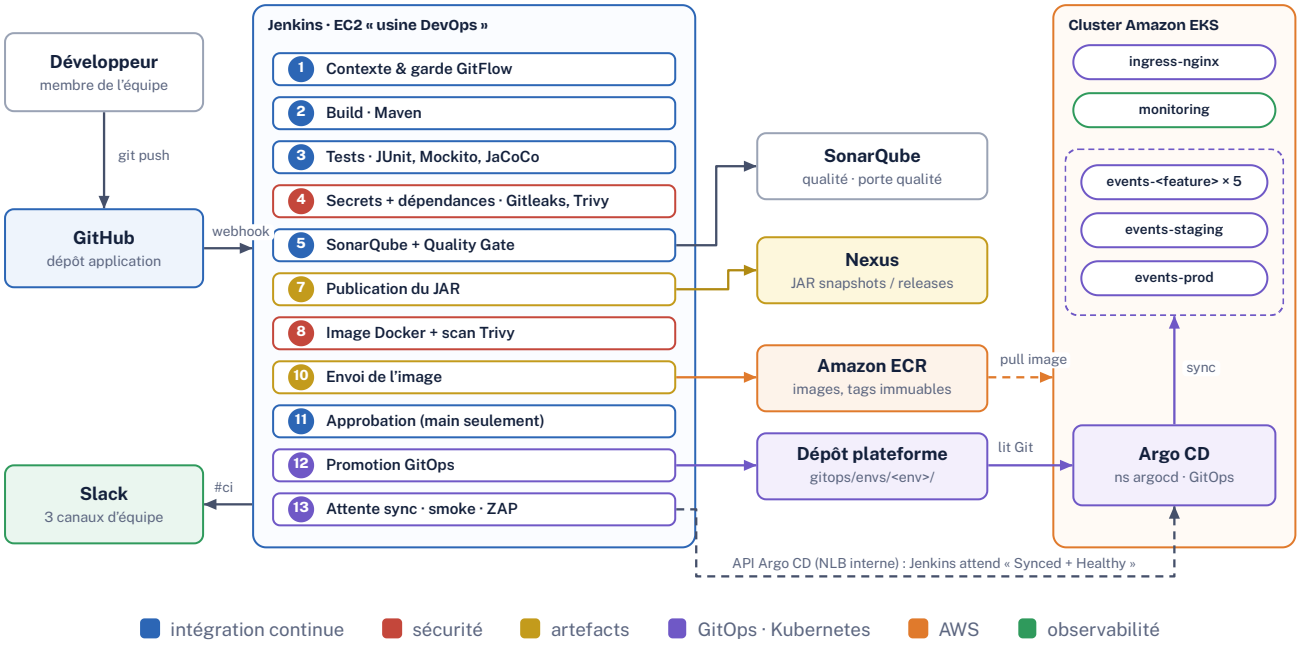


Figure 1. Vue d'ensemble. Jenkins orchestre l'intégration continue sur l'EC2 « usine DevOps ». Il publie le JAR dans Nexus et l'image dans ECR, puis écrit la nouvelle version dans le dépôt plateforme. Argo CD, dans le cluster, applique cette version.

Le parcours d'un commit

1. **Nour pousse sur** `feature/event`. GitHub prévient Jenkins par un webhook.
2. **Jenkins vérifie le code** : il déduit le contexte de la branche, compile, teste, cherche les secrets et les failles, fait analyser le code par SonarQube et attend le verdict de la porte qualité.
3. **Jenkins emballe ce qui a été validé** : le JAR part dans Nexus, l'image Docker est construite, scannée par Trivy et poussée dans Amazon ECR.
4. **Jenkins déploie… en écrivant dans Git** : il commite le nouveau tag d'image dans `gitops/envs/event/` du dépôt plateforme.
5. **Argo CD applique** : il détecte le commit, produit les ressources avec le chart Helm et met à jour le namespace `events-event`. Jenkins attend « Synced + Healthy », puis lance les tests de fumée et OWASP ZAP.
6. **La supervision prend le relais** : Prometheus découvre la nouvelle version, Grafana l'affiche, et Slack informe l'équipe à chaque étape clé.

5 Infrastructure AWS

Tout est hébergé dans la région AWS **eu-west-3 (Paris)**, la plus proche de Tunis. Le schéma se lit de l'extérieur vers l'intérieur : les points d'entrée, puis le réseau privé (VPC), ses deux zones de disponibilité, les sous-réseaux publics et privés, et enfin les services gérés par AWS.

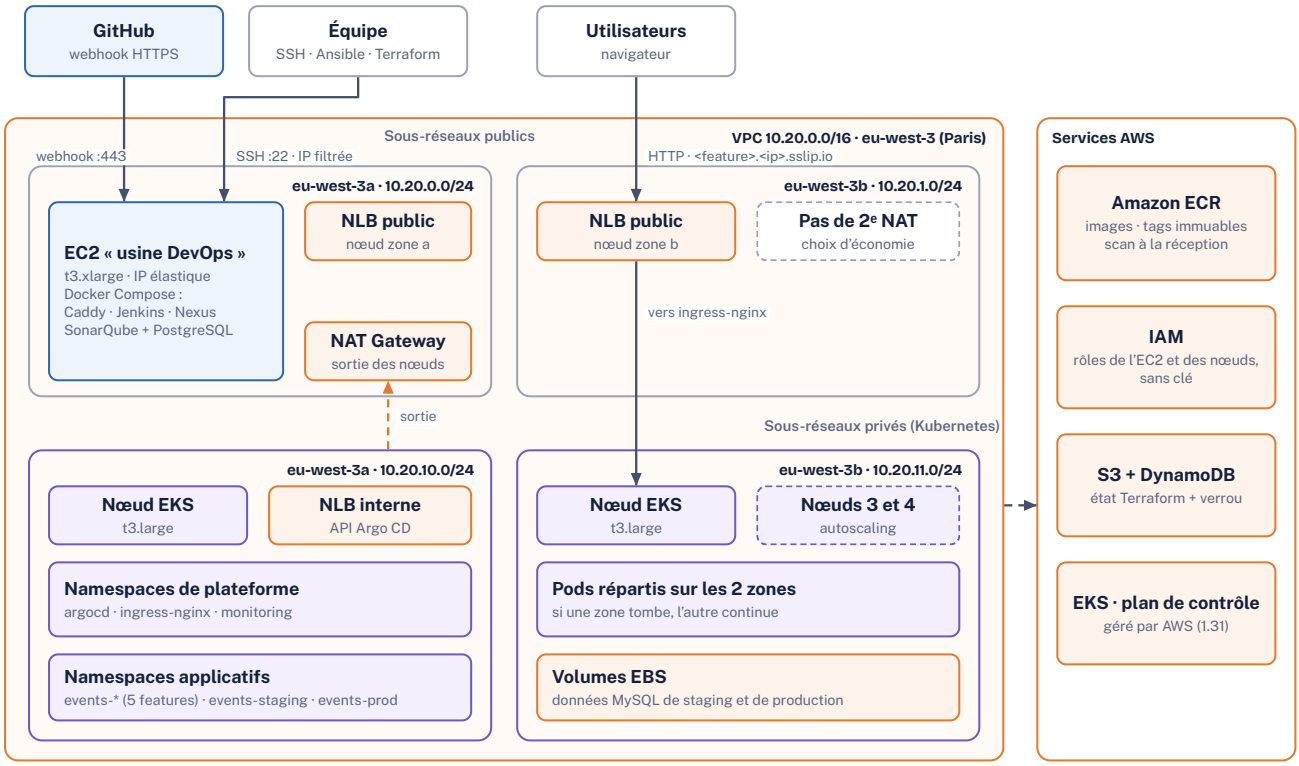


Figure 2. Topologie sur deux zones de disponibilité. Seuls le load balancer public et l'EC2 « usine DevOps » sont joignables depuis Internet. Les nœuds Kubernetes sont dans des sous-réseaux privés et sortent par la NAT.

5.1 Le réseau

Composant	Rôle	Pourquoi ce choix
VPC 10.20.0.0/16	Réseau privé et isolé de l'équipe dans AWS. Toutes les machines du projet y vivent.	Isolation complète des autres clients AWS. Plage d'adresses maîtrisée.
2 zones de disponibilité (a et b)	Deux centres de données physiquement séparés de la région de Paris.	Si une zone tombe, l'autre continue : les nœuds Kubernetes sont répartis sur les deux.
Sous-réseaux publics 10.20.0.0/24 et 10.20.1.0/24	Accueillent ce qui doit être joignable depuis Internet : le serveur des outils et le load balancer.	Exposer le strict minimum.
Sous-réseaux privés 10.20.10.0/24 et 10.20.11.0/24	Accueillent les nœuds Kubernetes, qui n'ont aucune adresse publique.	Un attaquant ne peut pas atteindre directement les serveurs applicatifs.
Internet Gateway	Porte d'entrée et de sortie Internet des sous-réseaux publics.	Indispensable pour les utilisateurs, GitHub et Let's Encrypt.
NAT Gateway	Permet aux nœuds privés de **sortir** vers Internet (lire GitHub, envoyer sur Slack) sans pouvoir être contactés de l'extérieur.	Une seule NAT au lieu de deux : deux fois moins chère, suffisant pour un projet étudiant.
Network Load Balancer public	Une seule adresse d'entrée pour toutes les features. Il transmet le trafic à l'ingress NGINX, qui oriente vers la bonne feature selon le nom d'hôte.	Un seul load balancer pour 7 environnements : économique et simple.

5.2 Les machines

EC2 « usine DevOps »

Instance **t3.xlarge** (4 vCPU, 16 Go) sous Ubuntu 24.04, avec une IP élastique fixe et un disque chiffré de 60 Go. Elle fait tourner, avec Docker Compose :

- **Caddy** : point d'entrée HTTPS, certificats Let's Encrypt automatiques ;
- **Jenkins** : orchestration du pipeline ;
- **Nexus** : dépôt des JAR ;
- **SonarQube** + **PostgreSQL** : analyse de qualité.

Cluster Amazon EKS « fst-events »

Kubernetes 1.31. Le **plan de contrôle** (API, etcd, ordonnanceur) est géré par AWS. Les **nœuds de calcul** sont 2 à 4 machines t3.large réparties sur les deux zones, qui exécutent :

- `argocd` : déploiement GitOps ;
- `ingress-nginx` : routage HTTP ;
- `monitoring` : Prometheus, Alertmanager, Grafana ;
- `events-*` : un namespace par feature, plus staging et prod.

5.3 Stockage et registres

Service	Ce qu'il stocke	Détail
Amazon ECR	Les images Docker de l'application.	Tags immuables, scan de sécurité à la réception, conservation des 50 dernières images.
Nexus (sur l'EC2)	Les JAR Maven.	*snapshots* pour les features, *releases* pour la production.
Volumes EBS	Les données MySQL de staging et de production.	Survivent au redémarrage des pods. Les features utilisent un disque temporaire, car leurs données de démo sont recréées.
S3 + DynamoDB	L'état Terraform (ce qui a été créé) et son verrou.	Deux membres ne peuvent pas modifier l'infrastructure en même temps.

5.4 Sécurité du réseau et des accès

Mesure	Effet
Groupe de sécurité de l'EC2	Seuls les ports 443 (HTTPS) et 80 (renouvellement des certificats) sont ouverts. Le port 22 (SSH) n'est accessible **que depuis l'IP de l'équipe**.
Rôles IAM	L'EC2 peut pousser dans ECR, et les nœuds peuvent lire ECR et utiliser les volumes EBS, grâce à des **rôles** attachés aux machines. **Aucune clé d'accès AWS n'est stockée**, ni dans Jenkins ni dans Git.
IMDSv2 obligatoire	Protège les identifiants temporaires de la machine contre une classe connue d'attaques (SSRF).
API Argo CD sur un NLB interne	Joignable uniquement depuis le VPC, donc par Jenkins. L'interface d'administration s'ouvre par un tunnel `kubectl port-forward`.
Nœuds privés	Aucune adresse publique : ils ne reçoivent du trafic qu'à travers le load balancer.

5.5 Les flux, dans l'ordre où ils se produisent

#	Flux	Ce qui circule et comment
1	GitHub → Jenkins	Webhook HTTPS (port 443) reçu par Caddy puis transmis à Jenkins : « la branche `feature/event` a changé ».
2	Jenkins → Amazon ECR	Envoi de l'image Docker, authentifié par le rôle IAM de l'EC2.
3	Jenkins → GitHub	Commit du nouveau tag d'image dans le dépôt plateforme (`gitops/envs/event/`).
4	Jenkins → Argo CD	Interrogation de l'API Argo CD (NLB interne) pour suivre l'avancement du déploiement.
5	Argo CD → GitHub	Lecture du dépôt plateforme. Sortie Internet par la NAT.
6	Nœuds EKS → Amazon ECR	Téléchargement de l'image grâce au rôle IAM des nœuds, sans aucun mot de passe.

#	Flux	Ce qui circule et comment
7	Utilisateurs → NLB → ingress → pod	Requête HTTP sur `event.<ip>.sslip.io/events/`. L'ingress lit le nom d'hôte et envoie vers le service de la feature *event*.
8	Alertmanager, Argo CD, Jenkins → Slack	Messages vers les trois canaux de l'équipe.
T	Terraform → API AWS	Création et mise à jour de toutes les ressources (une seule fois, puis à chaque évolution de l'infrastructure).
A	Ansible → EC2	Connexion SSH (port 22) depuis le poste de l'équipe pour installer et configurer les outils.

À retenir

Seuls deux points sont exposés à Internet : le load balancer, pour les applications, et l'EC2, pour les outils en HTTPS et le SSH limité à une IP. Tout le reste est privé. Les machines s'authentifient auprès d'AWS par leurs rôles, jamais par des clés écrites quelque part.

6 Infrastructure as Code : Terraform, Ansible, Argo CD

Aucune ressource n'est créée à la main dans la console AWS. Trois outils se succèdent, chacun dans son domaine : **Terraform crée** les ressources AWS, **Ansible configure** le serveur des outils, et **Argo CD déploie** tout ce qui tourne dans le cluster.

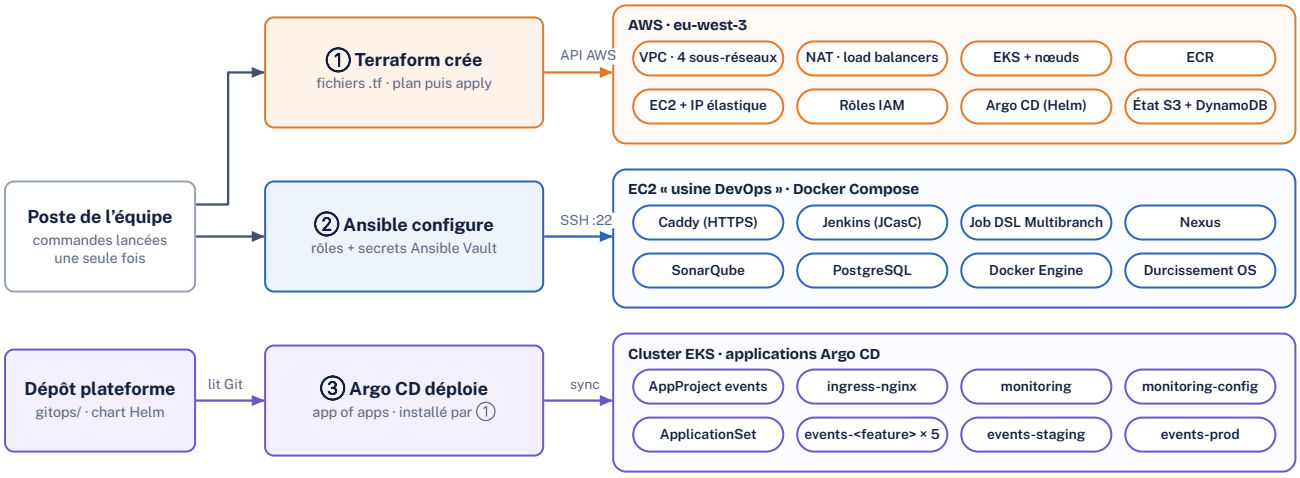


Figure 3. Qui crée quoi. Terraform et Ansible sont lancés depuis le poste de l'équipe. Argo CD, installé par Terraform, lit ensuite le dépôt plateforme et déploie seul le reste.

① Terraform crée

Il décrit dans des fichiers `.tf` l'infrastructure voulue, la compare à ce qui existe (son « état ») et crée, modifie ou supprime ce qui doit l'être.

② Ansible configure

Il se connecte en SSH au serveur EC2 et y installe les logiciels de façon **idempotente** : relancé dix fois, le résultat reste le même.

③ Argo CD déploie

Installé par Terraform, il prend le relais dans le cluster et y déploie tout le reste à partir de Git, en continu.

6.1 Terraform : ce qu'il crée, fichier par fichier

Fichier	Ressources créées	Pourquoi
`network.tf`	VPC, 2 sous-réseaux publics, 2 privés, Internet Gateway, NAT, tables de routage (module officiel *terraform-aws-modules/vpc*)	Réseau isolé sur deux zones.
`eks.tf`	Cluster EKS 1.31, groupe de nœuds 2 à 4 × t3.large, modules complémentaires CoreDNS, VPC CNI et pilote EBS (module *terraform-aws-modules/eks*)	Kubernetes géré par AWS, avec autoscaling.
`ecr.tf`	Registre `events-app` (tags immuables, scan, politique de rétention)	Stockage des images.
`devops-ec2.tf`	Instance EC2, IP élastique, clé SSH, groupe de sécurité, rôle et profil IAM	Serveur des outils, moindre privilège.
`argocd.tf`	Argo CD (chart Helm) + application racine, namespace et secrets du monitoring	Amorçage du GitOps.
`ansible-handoff.tf`	Inventaire Ansible (IP de l'EC2) et variables (registre ECR, région)	Passage de relais automatique.

Commande	Effet
`terraform init`	Télécharge fournisseurs et modules, se connecte à l'état partagé (S3).
`terraform plan`	Affiche ce qui va être créé, modifié ou détruit, **sans rien toucher**. C'est la relecture avant d'agir.
`terraform apply`	Applique le plan : environ 20 minutes la première fois, principalement pour le cluster EKS.
`terraform destroy`	Supprime tout, ce qui évite de payer entre deux séances.

6.2 Ansible : ce qu'il installe sur l'EC2

Rôle Ansible	Actions
`common`	Paquets de base, fuseau horaire Africa/Tunis, réglages du noyau exigés par SonarQube, mises à jour de sécurité automatiques.
`docker`	Docker Engine et Docker Compose, rotation des journaux des conteneurs.
`devops_stack`	Génère et lance la pile Docker Compose : Caddy, Jenkins, Nexus, SonarQube, PostgreSQL. Configure Jenkins **entièrement par du code** (*Jenkins Configuration as Code*) : comptes, identifiants, variables globales, serveur SonarQube, notifications Slack. Le job Multibranch lui-même est créé par **Job DSL**.
`nexus_config`	Remplace le mot de passe initial de Nexus et désactive l'accès anonyme, via l'API REST de Nexus.

Mots de passe et jetons sont chiffrés avec **Ansible Vault** : versionnés, jamais en clair.

6.3 Argo CD : le modèle « app of apps »

Terraform n'installe qu'Argo CD et **une application racine**, qui pointe vers un dossier du dépôt plateforme décrivant toutes les autres applications. Argo CD les installe alors lui-même :

Application	Contenu
AppProject « events »	Règle de sécurité : les environnements Events ne peuvent être déployés que dans des namespaces `events-*`.
ingress-nginx	Routeur HTTP du cluster et son load balancer public.
monitoring	Prometheus, Alertmanager et Grafana (kube-prometheus-stack).
monitoring-config	Règles d'alerte de l'application et dashboard Grafana « Events FST · par feature ».

Application	Contenu
ApplicationSet « events-environments »	Générateur d'environnements : **chaque dossier `gitops/envs/<env>/` devient une application `events-<env>`**. C'est lui qui crée automatiquement l'environnement d'une nouvelle feature.

6.4 Ordre de mise en place (une seule fois, environ 30 minutes)

1. **Terraform** crée le réseau, le cluster, le registre, le serveur et les droits, puis installe Argo CD.
2. **Ansible** configure le serveur : Jenkins, Nexus et SonarQube démarrent, déjà configurés.
3. **Argo CD** déploie seul l'ingress et le monitoring. Chaque environnement apparaît au premier build de sa branche.
4. On crée trois jetons (Argo CD, SonarQube, GitHub) et deux webhooks (GitHub → Jenkins, SonarQube → Jenkins), puis on relance Ansible pour les injecter dans Jenkins.

7 Pipeline CI/CD : les étapes en un coup d'œil

Le pipeline est décrit dans le `Jenkinsfile`, versionné avec le code. Il compte 17 étapes réparties en trois phases. Chaque carte indique la **technologie** qui réalise l'étape et la **condition qui l'arrête**.

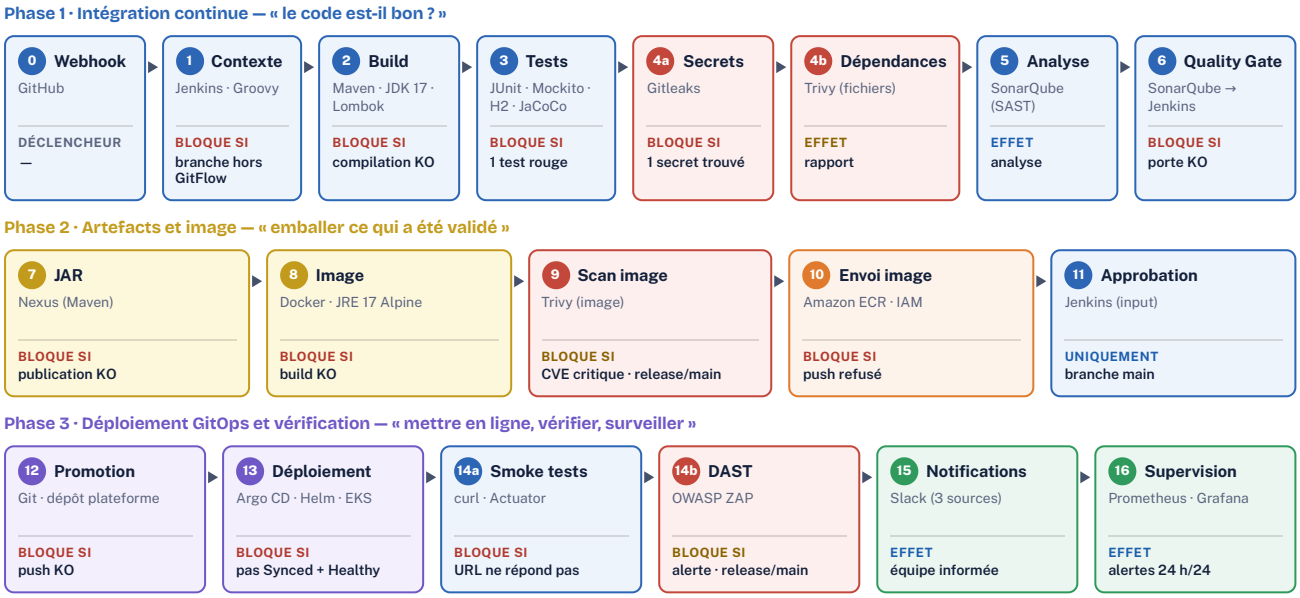


Figure 4. Les étapes du pipeline. En rouge : les contrôles de sécurité ; en jaune : les artefacts ; en violet : le déploiement GitOps. Une étape « Bloque si » arrête tout le pipeline dès que la condition est remplie.

> **Jenkins est le chef d'orchestre.** Il ne fait presque rien lui-même : il lit le `Jenkinsfile` et appelle chaque outil dans l'ordre. Dès qu'une étape BLOQUANTE échoue, il s'arrête et rien de ce qui suit ne s'exécute. Une étape en RAPPORT publie ses résultats sans arrêter le pipeline, sauf sur `release` et `main`.

Récapitulatif : chaque étape, sa technologie, ce qu'elle produit

#	Étape	Technologie	Produit	Effet
0	Déclenchement	GitHub (webhook)	build #n lancé	—
1	Contexte & GitFlow	Jenkins, Groovy, Git	responsable, version, environnement, tag	BLOQUANT
2	Build	Maven, JDK 17, Lombok	classes compilées	BLOQUANT
3	Tests & couverture	JUnit, Mockito, H2, JaCoCo	JAR, rapports de tests et de couverture	BLOQUANT
4a	Secrets	Gitleaks	rapport de secrets	BLOQUANT
4b	Dépendances	Trivy (fichiers)	rapport de CVE	RAPPORT
5	Qualité	SonarQube	analyse du projet de la feature	ANALYSE
6	Quality Gate	SonarQube → Jenkins	verdict OK / KO	BLOQUANT
7	Publication JAR	Nexus	JAR versionné archivé	BLOQUANT
8	Image	Docker	image étiquetée	BLOQUANT
9	Scan image	Trivy (image)	rapport de CVE de l'image	RAPPORT
10	Envoi image	Amazon ECR, AWS CLI, IAM	image dans le registre	BLOQUANT
11	Approbation	Jenkins (input)	feu vert humain (main)	MAIN
12	Promotion GitOps	Git (dépôt plateforme)	commit de déploiement	BLOQUANT

#	Étape	Technologie	Produit	Effet
13	Déploiement	Argo CD, Helm, Kubernetes EKS	environnement en ligne	BLOQUANT
14a	Smoke tests	curl, Spring Actuator	preuve de fonctionnement	BLOQUANT
14b	DAST	OWASP ZAP	rapport de sécurité dynamique	RAPPORT
15	Notifications	Slack	équipe informée	—
16	Supervision	Prometheus, Alertmanager, Grafana	santé visible, alertes	—

8 Les étapes du pipeline expliquées une par une

Chaque fiche indique la **technologie** qui réalise l'étape et **ce qui se passe** concrètement. L'encadré gris donne ce que cela donne dans notre exemple (le build n° 12 de Nour sur `feature/event`), le **résultat** produit et ce qui arrive **si l'étape échoue**.

Phase 1 · Intégration continue

étapes 0 à 6 · « le code est-il bon ? »

0 Déclenchement

GitHub · webhook

Le membre pousse ses commits sur sa branche. GitHub envoie aussitôt un **webhook**, un message HTTPS, à l'adresse de Jenkins pour signaler qu'une branche a changé.

Jenkins, grâce au job *Multibranch*, retrouve la branche correspondante (ou la découvre si elle est nouvelle), récupère le code et démarre un nouveau build. Les builds d'une même branche ne se chevauchent pas : le suivant attend la fin du précédent.

DANS NOTRE EXEMPLE

Nour pousse sur `feature/event`. Le build `#12 · event` démarre en quelques secondes.

RÉSULTAT

Un espace de travail Jenkins contenant le code du commit `a1b2c3d`.

SI ÇA ÉCHOUE

Webhook non reçu : Jenkins vérifie de toute façon les branches toutes les 5 minutes.

1 Contexte et garde GitFlow

Jenkins · Groovy · Git BLOQUANT

Jenkins lit le nom de la branche et en déduit toutes les valeurs dont les étapes suivantes ont besoin : le **responsable** (à partir d'une table de correspondance), le **type de branche**, la **version Maven**, l'**environnement** cible, le **tag de l'image** (feature, numéro de build, identifiant du commit), la **clé du projet SonarQube**, l'**URL** et le niveau de sévérité des contrôles de sécurité.

Il vérifie aussi que la branche respecte GitFlow (`feature/…` , `release/x.y.z` , `hotfix/…`), et qu'une Pull Request de feature vise bien `develop` . C'est cette étape qui permet au même pipeline de servir aux cinq membres.

DANS NOTRE EXEMPLE

Responsable : Nour Mustapha · version `1.0.0-event-SNAPSHOT` · environnement `events-event` · image `event-12-a1b2c3d` · Sonar `fst-events-event` .

RÉSULTAT

Le build est renommé « #12 · event » avec Nour en description.

SI ÇA ÉCHOUE

Branche mal nommée (ex. `Feature_Event`) : arrêt immédiat avec un message explicite.

2 Build

Maven · JDK 17 · Lombok | BLOQUANT

Maven lit le `pom.xml`, télécharge les dépendances (Spring Boot, MySQL, Lombok…) et applique la **version propre à la feature**. Sans cela, les cinq membres publieraient le même numéro de version et s'écraseraient mutuellement dans Nexus.

Il **compile** ensuite le code Java avec le JDK 17. Au passage, **Lombok** génère automatiquement les getters, setters, constructeurs et loggers déclarés par annotations.

DANS NOTRE EXEMPLE
La version passe à `1.0.0-event-SNAPSHOT` et le code de Nour est compilé.

RÉSULTAT
Les classes compilées dans `target/classes`.

SI ÇA ÉCHOUE
Erreur de compilation : arrêt, et l'erreur exacte (fichier, ligne) apparaît dans la console Jenkins.

3 Tests et couverture

JUnit · Mockito · H2 · JaCoCo | BLOQUANT

Maven exécute toute la suite de tests, de deux natures différentes :

- **Tests avec Mockito** : on teste un service seul en remplaçant ses dépendances (les repositories) par des objets simulés. Ils sont rapides et vérifient la logique métier, par exemple le calcul des statistiques ou le refus d'un intervalle de dates invalide.
- **Tests sans Mockito (intégration)** : Spring démarre réellement, avec une base **H2 en mémoire** à la place de MySQL. On vérifie que contrôleurs, services, repositories et base fonctionnent ensemble.

JaCoCo mesure pendant ce temps quelles lignes ont été exécutées (la couverture). Maven assemble enfin le **JAR exécutable**.

DANS NOTRE EXEMPLE
Les tests de Nour et ceux de toute l'équipe passent, ce qui garantit qu'il n'a rien cassé chez les autres.

RÉSULTAT
Le JAR, le rapport de tests et le rapport de couverture, affichés dans Jenkins.

SI ÇA ÉCHOUE
Un seul test rouge suffit à tout arrêter. Jenkins affiche le test fautif et sa trace.

4a Recherche de secrets

Gitleaks | BLOQUANT

Exécuté **en parallèle** de l'étape 4b. Gitleaks parcourt **tout l'historique Git**, pas seulement la dernière version, à la recherche de mots de passe, jetons d'API, clés privées ou clés AWS. Il s'appuie sur des centaines de motifs connus et sur la mesure du caractère aléatoire des chaînes.

C'est bloquant sur toutes les branches : un secret poussé sur GitHub doit être considéré comme compromis, même s'il est supprimé ensuite, puisqu'il reste dans l'historique.

RÉSULTAT
Un rapport JSON archivé dans Jenkins, avec les valeurs masquées.

SI ÇA ÉCHOUE
Arrêt. Il faut révoquer le secret, nettoyer l'historique, puis repousser.

4b Analyse des dépendances

Trivy (mode fichiers) | RAPPORT

Trivy liste toutes les librairies du projet, y compris les dépendances indirectes, et les compare à une base de **failles connues (CVE)** : c'est l'analyse de composition logicielle (**SCA**). Il contrôle aussi les erreurs de configuration du `Dockerfile`, par exemple une image qui tournerait en root.

Le résultat est publié sans bloquer, parce que Spring Boot 2.7.6 (hors support) contient des failles connues qui ne viennent pas du code de l'équipe.

RÉSULTAT
Le rapport `trivy-fs.txt` : librairie, faille, gravité et version corrigée.

ACTION
Il sert de liste de travail pour la montée de version (voir la section 14).

5 Analyse statique de la qualité

SonarQube · ANALYSE

Le plugin Maven de SonarQube envoie le code source, les classes compilées et le rapport JaCoCo au serveur SonarQube. Celui-ci **lit le code sans l'exécuter** (**SAST**) et relève :

- les **bugs** probables ;
- les **vulnérabilités** et points sensibles de sécurité ;
- les *code smells* : code difficile à maintenir ;
- les **duplications** et la **couverture** par les tests.

Chaque feature a son propre projet SonarQube, ce qui permet de suivre la qualité du travail de chaque membre séparément.

DANS NOTRE EXEMPLE
Le projet `fst-events-event` est créé au premier build de Nour, puis mis à jour à chaque build.

RÉSULTAT
Un tableau de bord SonarQube avec l'historique de la feature.

6 Porte qualité (Quality Gate)

SonarQube → webhook → Jenkins · BLOQUANT

SonarQube compare les résultats à des seuils (par exemple : aucun nouveau bug critique, couverture suffisante du nouveau code). Il renvoie son verdict à Jenkins par un **webhook**, et Jenkins attend ce verdict avant de continuer.

C'est la frontière entre **intégration** et **livraison** : du code qui ne passe pas la porte qualité ne produit ni artefact ni déploiement. Pour une Pull Request, le pipeline s'arrête ici avec succès : on a vérifié le code sans rien déployer.

RÉSULTAT
« OK » : le pipeline passe à la livraison.

SI ÇA ÉCHOUE
« KO » : arrêt. Le lien vers SonarQube montre précisément les problèmes à corriger.

Phase 2 · Artefacts et image

étapes 7 à 11 · « emballer ce qui a été validé »

7 Publication du JAR

Nexus (Maven) · BLOQUANT

Maven envoie le JAR testé dans **Nexus**, le dépôt d'artefacts privé de l'équipe. Les identifiants sont injectés par Jenkins au moment de l'étape, et n'apparaissent jamais dans Git.

- Les features et `develop` vont dans `maven-snapshots` (versions de travail).
- `main` va dans `maven-releases` (versions définitives, numérotées `1.0.<build>`).

On peut ainsi retrouver à tout moment le binaire exact correspondant à une version.

DANS NOTRE EXEMPLE
`eventsProject-1.0.0-event-SNAPSHOT.jar` dans `maven-snapshots`.

SI ÇA ÉCHOUE
Nexus indisponible ou identifiants invalides : arrêt.

8 Construction de l'image

Docker · JRE 17 Alpine · BLOQUANT

Docker suit le `Dockerfile` : il part d'une image de base légère (**Java 17, version d'exécution seule, sur Alpine Linux**), crée un utilisateur sans privilèges et y **copie le JAR déjà testé**, sans le recompiler. C'est le principe *build once, deploy many* : ce qui sera déployé est exactement ce qui a été testé.

L'image reçoit des étiquettes : feature, responsable, version et commit. La JVM est réglée pour s'adapter automatiquement à la mémoire allouée par Kubernetes.

DANS NOTRE EXEMPLE
`events-app:event-12-a1b2c3d`, étiquetée `owner=nour-mustapha`.

RÉSULTAT
Une image d'environ 200 Mo, sans compilateur ni outils inutiles.

9 Scan de l'image

Trivy (mode image) · RAPPORT · STRICT SUR RELEASE/MAIN

Trivy analyse cette fois l'image complète : les paquets du système Alpine et toutes les librairies Java embarquées. Il ne retient que les failles **critiques** pour lesquelles un correctif existe, c'est-à-dire celles sur lesquelles on peut agir.

Sur une feature, le résultat est un rapport. Sur `release` et `main`, une telle faille **bloque la mise en production**.

RÉSULTAT
Le rapport `trivy-image.txt`, archivé dans Jenkins.

SI ÇA ÉCHOUE
Sur `main` : mettre à jour la librairie ou l'image de base, puis relancer.

10 Envoi de l'image

Amazon ECR · AWS CLI · IAM · BLOQUANT

Jenkins obtient un jeton de connexion temporaire auprès d'AWS grâce au **rôle IAM de l'EC2**, sans aucune clé stockée, puis pousse l'image dans **Amazon ECR**.

Le registre est configuré avec des **tags immuables** : `event-12-a1b2c3d` désignera toujours exactement cette image. ECR effectue aussi son propre **scan de sécurité à la réception**, une seconde vérification indépendante de Trivy.

RÉSULTAT
L'image est disponible pour le cluster EKS.

SI ÇA ÉCHOUE
Droits IAM insuffisants ou tag déjà existant : arrêt.

11 Approbation de la production

Jenkins (étape « input ») · MAIN UNIQUEMENT

Uniquement pour la branche `main` : le pipeline se met en pause et affiche un bouton « Promouvoir ». Une personne doit valider explicitement la mise en production (délai maximum : 24 heures). Pour toutes les autres branches, l'étape est sautée.

C'est le seul geste humain de toute la chaîne. Il place une **décision consciente** entre le code validé et les utilisateurs réels.

DANS NOTRE EXEMPLE
Sautée : `feature/event` n'est pas `main`.

SI REFUSÉE
Le build s'arrête. L'image reste dans ECR, prête à être promue plus tard.

Phase 3 · Déploiement GitOps et vérification

étapes 12 à 16 · « mettre en ligne, vérifier, surveiller »

12 Promotion GitOps

Git · dépôt plateforme · BLOQUANT

Jenkins ne déploie pas lui-même. Il clone le dépôt plateforme et écrit, dans le dossier de l'environnement (`gitops/envs/event/`), deux petits fichiers : la configuration de l'environnement (namespace, profil, responsable) et ses valeurs (dépôt et **tag de l'image**, URL).

Il pousse ce commit sur `main` du dépôt plateforme. Si un autre membre a poussé au même moment, Jenkins se resynchronise et réessaie (jusqu'à 5 fois). Git devient la **source de vérité** : ce qui y est écrit est ce qui doit tourner.

DANS NOTRE EXEMPLE
Commit « deploy(event): event-12-a1b2c3d [build #12 · Nour Mustapha] ».

RÉSULTAT
L'état voulu de `events-event` est à jour dans Git.

RETOUR ARRIÈRE
Annuler ce commit (`git revert`) suffit à revenir à la version précédente.

13 Synchronisation et déploiement

Argo CD → Helm → Kubernetes (EKS) · BLOQUANT

Tout se passe dans le cluster, sans Jenkins (le détail est en section 9) :

- **Argo CD** détecte le nouveau commit dans le dépôt plateforme.
- Pour une feature nouvelle, son **ApplicationSet** crée automatiquement l'application `events-event` et son namespace.
- Argo CD utilise le **chart Helm** pour produire les ressources Kubernetes : l'application, sa base **MySQL** (avec un mot de passe généré une seule fois), son service, son ingress et son *ServiceMonitor* pour Prometheus.
- **Kubernetes** télécharge l'image depuis ECR et remplace les pods **un par un** (*rolling update*) : l'ancienne version sert les requêtes jusqu'à ce que la nouvelle soit prête.
- Les **sondes Spring Actuator** (démarrage, vie, disponibilité) indiquent quand un pod peut recevoir du trafic.

Pendant ce temps, Jenkins interroge Argo CD toutes les 10 secondes. Il attend que **sa révision exacte** soit appliquée, avec l'état **Synced** (le cluster est conforme à Git) et **Healthy** (les pods fonctionnent).

DANS NOTRE EXEMPLE
Le namespace `events-event` contient le pod de l'application et le pod MySQL, avec des données de démo.

RÉSULTAT
« Synced + Healthy » en 1 à 3 minutes.

SI ÇA ÉCHOUE
État
DEGRADED
(ex. le pod plante au démarrage) ou délai de 10 minutes dépassé : arrêt. L'interface Argo CD montre les événements et les journaux des pods.

14a Tests de fumée (smoke tests)

curl · Spring Actuator | BLOQUANT

Exécuté **en parallèle** de l'étape 14b. Jenkins appelle la **vraie URL publique** de la feature, en passant par le load balancer et l'ingress, comme le ferait un utilisateur :

- l'état de santé : `/events/actuator/health`, avec plusieurs essais le temps du démarrage ;
- l'API métier : `/events/event-stats/summary` ;
- la page du tableau de bord.

On prouve ainsi que l'application **fonctionne réellement** de bout en bout (réseau, base, code), pas seulement qu'elle a démarré.

DANS NOTRE EXEMPLE

`http://event.<ip>.sslip.io/events/` répond « UP ».

SI ÇA ÉCHOUE

Arrêt : le déploiement est en ligne mais défectueux, à corriger ou à annuler par `git revert` .

14b Test de sécurité dynamique (DAST)

OWASP ZAP (baseline) | RAPPORT · STRICT SUR RELEASE/MAIN

OWASP ZAP explore l'application en marche (pendant 2 minutes) et l'analyse comme le ferait un attaquant, mais de façon **passive**, donc sans danger :

- en-têtes de sécurité HTTP manquants ;
- cookies mal protégés ;
- fuites d'information dans les erreurs.

C'est le seul contrôle qui voit l'application **en exécution**, avec sa configuration réelle.

RÉSULTAT

Une liste d'alertes classées par risque, visible dans la console Jenkins.

SUR RELEASE/MAIN

Une alerte de niveau « échec » bloque la livraison.

15 Notifications

Slack (Jenkins · Argo CD · Alertmanager)

Trois sources différentes publient chacune dans **leur propre canal**, pour que l'équipe sache immédiatement d'où vient l'information :

- **#igl5-g2-ci** : Jenkins publie le résultat de chaque build (succès ou échec, feature, responsable, lien).
- **#igl5-g2-deploiements** : **Argo CD Notifications** publie la version déployée, un échec de synchronisation ou une santé dégradée.
- **#igl5-g2-alertes** : **Alertmanager** publie les incidents en exécution, puis leur résolution (étape 16).

DANS NOTRE EXEMPLE

#ci : « SUCCESS · event (#12) · Nour Mustapha ». #deploiements : « events-event déployée — image event-12-a1b2c3d ».

16 Supervision continue

Prometheus · Alertmanager · Grafana

Le pipeline est terminé, mais la surveillance ne s'arrête jamais. **Prometheus** découvre automatiquement la nouvelle version grâce au *ServiceMonitor* et relève ses métriques toutes les 30 secondes. **Alertmanager** transmet les alertes éventuelles à l'équipe. **Grafana** permet à Nour de suivre sa feature dans le dashboard « Events FST · par feature ». Le fonctionnement détaillé est décrit en section 10.

RÉSULTAT

La santé de chaque feature est visible 24 heures sur 24.

9 Plateforme Kubernetes

Le cluster EKS héberge à la fois les outils de plateforme et les environnements de l'application. Chaque environnement est un **namespace**, produit par le **même chart Helm** avec un profil différent. Argo CD maintient le cluster aligné sur le dépôt plateforme.

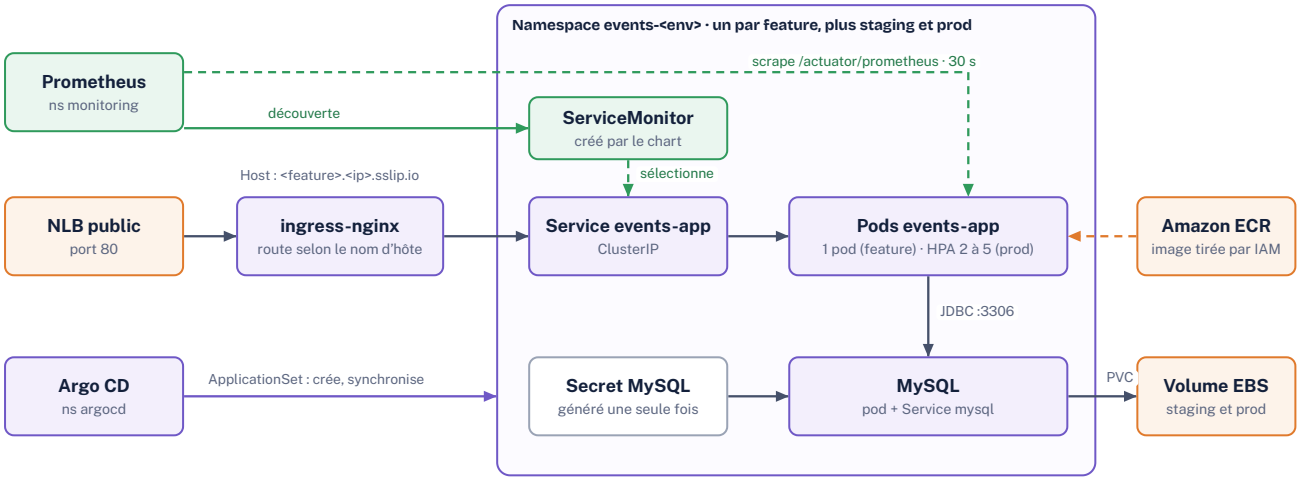


Figure 5. Contenu d'un environnement et flux autour de lui. Le trafic entre par le NLB et ingress-nginx ; Prometheus découvre l'application grâce au ServiceMonitor ; Argo CD crée et synchronise le namespace. En vert : la supervision ; en pointillés orange : le téléchargement de l'image.

9.1 Les namespaces du cluster

Namespace	Contenu	Profil
`argocd`	Argo CD, l'ApplicationSet et Argo CD Notifications. L'API est exposée par un NLB interne.	Plateforme
`ingress-nginx`	Le contrôleur ingress et son load balancer public : l'unique entrée HTTP du cluster.	Plateforme
`monitoring`	Prometheus, Alertmanager, Grafana, kube-state-metrics et node-exporter (kube-prometheus-stack).	Plateforme
`events-<feature>` × 5	Un environnement par membre : application, MySQL, service, ingress, ServiceMonitor.	Review : 1 pod, données de démo, disque temporaire
`events-staging`	Intégration de `develop` puis recette des branches `release`.	Staging : MySQL sur volume EBS
`events-prod`	Production, alimentée par `main` après approbation.	Prod : HPA 2 à 5 pods, MySQL sur EBS

9.2 Ce que le chart Helm crée dans chaque environnement

Ressource	Rôle et réglages
Deployment events-app	Les pods de l'application, mis à jour par *rolling update*. Sondes Actuator de démarrage, de vie et de disponibilité. Ressources CPU et mémoire réservées et plafonnées.
Service et Ingress	Le Service donne une adresse stable aux pods. L'Ingress relie le nom d'hôte `<feature>.<ip>.sslip.io` au Service.
MySQL et son Secret	Un pod MySQL et son service. Le mot de passe est généré une seule fois et conservé dans un Secret Kubernetes, jamais écrit dans Git.
ServiceMonitor	Indique à Prometheus où relever les métriques : `/events/actuator/prometheus`, toutes les 30 secondes.

Ressource	Rôle et réglages
HorizontalPodAutoscaler	En production seulement : de 2 à 5 pods selon la charge CPU.
Contexte de sécurité	Pods non-root, système de fichiers en lecture seule, `capabilities: drop ALL`, profil seccomp.

> **Ce que nous n'avons pas encore :** des **NetworkPolicies**. Aujourd'hui, tous les pods du cluster peuvent se joindre entre namespaces. Une politique « tout refuser sauf les flux prévus » est la première amélioration de sécurité réseau proposée en section 14.

10 Observabilité et système de notification

Trois outils travaillent ensemble, mais chacun a un rôle bien distinct. C'est souvent ce qui est mal compris : **Prometheus détecte**, **Alertmanager prévient**, **Grafana montre**. Le schéma montre aussi les deux autres sources de messages Slack : Jenkins et Argo CD.

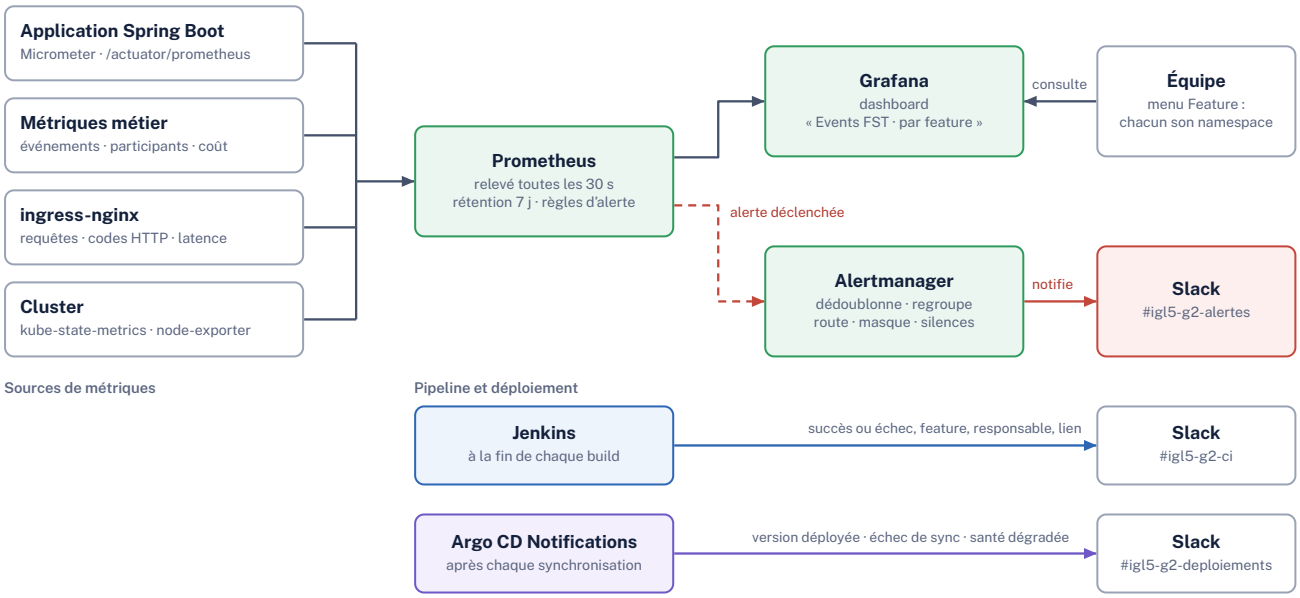


Figure 6. Chaîne d'observabilité, déployée avec kube-prometheus-stack. En rouge : le chemin d'une alerte. En bas : les notifications du pipeline et des déploiements.

10.1 Qui fait quoi

Outil	Rôle	Dans notre projet
Micrometer (dans l'application)	Exposer les mesures	L'application publie sur `/events/actuator/prometheus` : nombre et durée des requêtes HTTP par URL, erreurs, mémoire et CPU de la JVM, connexions à MySQL, et trois **métriques métier** (nombre d'événements, de participants, coût de la logistique réservée).
Prometheus	**Collecter et détecter**	Trouve chaque environnement grâce au *ServiceMonitor* créé par le chart Helm, relève les métriques toutes les 30 secondes, les conserve 7 jours et **évalue en continu les règles d'alerte**. C'est lui qui décide qu'il y a un problème.
Alertmanager	**Trier et prévenir**	Ne mesure rien. Il **reçoit** les alertes de Prometheus, **supprime les doublons**, les **regroupe** par namespace, les **route** selon leur gravité, **masque** les alertes secondaires (si l'application est arrêtée, inutile d'alerter aussi sur sa latence), gère les **silences** pendant une maintenance et envoie le message de **résolution**. C'est le « standard téléphonique » des incidents.
Grafana	**Visualiser**	Interroge Prometheus et affiche le dashboard « Events FST · par feature ». Un menu **Feature** permet à chacun de choisir son namespace.

10.2 Les règles d'alerte

Alerte	Se déclenche quand…	Gravité
`EventsAppDown`	l'application ne répond plus à Prometheus depuis plus de 2 minutes	CRITIQUE
`EventsHighErrorRate`	plus de 5 % des requêtes renvoient une erreur 5xx pendant 5 minutes	AVERTISSEMENT
`EventsHighLatencyP95`	5 % des requêtes mettent plus d'une seconde, pendant 10 minutes	AVERTISSEMENT
`EventsJvmHeapHigh`	la mémoire Java dépasse 85 % pendant 10 minutes	AVERTISSEMENT

10.3 Le dashboard « Events FST · par feature »

DISPONIBILITÉ
Statut
UP ou DOWN de l'environnement choisi, et historique des coupures.

TRAFIC
Requêtes / s
Débit HTTP par URL : quelles API sont réellement utilisées.

ERREURS
Taux de 5xx
Part des requêtes en erreur, à comparer au seuil de 5 %.

PERFORMANCE
Latence
p50, p95 et p99 : l'expérience typique et celle des plus lents.

JVM
Mémoire et CPU
Tas Java utilisé et ramasse-miettes, à surveiller avant le seuil de 85 %.

BASE
Connexions MySQL
Connexions actives et en attente dans le pool.

MÉTIER
Indicateurs Events
Nombre d'événements, de participants et coût de la logistique réservée.

FILTRE
Menu Feature
Chaque membre sélectionne son namespace `events-<feature>`.

10.4 Le système de notification

Canal Slack	Source	Contenu
`#igl5-g2-ci`	Jenkins	Résultat de chaque build : succès ou échec, feature, responsable, lien.
`#igl5-g2-deploiements`	Argo CD Notifications	Version déployée (image et révision), échec de synchronisation, santé dégradée.
`#igl5-g2-alertes`	Alertmanager	Incidents en exécution, critiques ou avertissements, puis « RESOLVED ». Un envoi par e-mail est possible en plus.

Un incident, de bout en bout

Le pod de `events-event` plante. Prometheus ne parvient plus à relever ses métriques (`up == 0`) et, au bout de 2 minutes, la règle `EventsAppDown` se déclenche. Alertmanager la classe **critique**, masque les alertes de latence du même namespace et publie sur `#igl5-g2-alertes` : « CRITIQUE · EventsAppDown · events-event ». Pendant ce temps, Kubernetes redémarre automatiquement le pod. Dès qu'il répond de nouveau, Alertmanager publie « RESOLVED », et Grafana montre la coupure sur la courbe de disponibilité.

11 Sécurité de bout en bout (DevSecOps)

La sécurité n'est pas une étape ajoutée à la fin : elle est intégrée à **chaque niveau**, le plus tôt possible (*shift-left*), là où un problème coûte le moins cher à corriger.

Niveau	Mesure	Ce qu'elle protège
Code	Gitleaks	Fuite de mots de passe ou de jetons dans l'historique Git.
Code	SonarQube (SAST)	Failles dans le code écrit par l'équipe : injections, mauvaises pratiques.
Dépendances	Trivy FS (SCA)	Failles connues dans les librairies utilisées.

Niveau	Mesure	Ce qu'elle protège
Image	Trivy image + scan ECR	Failles du système de base et des librairies embarquées. Double vérification.
Image	Utilisateur non-root, tags immuables	Limite les dégâts en cas de compromission. Empêche le remplacement silencieux d'une version.
Exécution	OWASP ZAP (DAST)	Failles visibles uniquement en marche : en-têtes, cookies, erreurs bavardes.
Pods	Non-root, système de fichiers en lecture seule, `capabilities: drop ALL`, seccomp	Un attaquant qui exploiterait l'application ne pourrait presque rien faire dans le conteneur.
Déploiement	GitOps, AppProject, API Argo CD interne	Jenkins n'a aucun droit sur le cluster. Argo CD ne peut déployer que dans `events-*`.
Secrets	Ansible Vault, identifiants Jenkins, secrets Kubernetes	Aucun secret en clair dans Git. Le mot de passe MySQL est généré automatiquement.
Cloud	Rôles IAM, IMDSv2, disques chiffrés, SSH limité à une IP, nœuds privés	Pas de clé AWS à voler. Surface d'attaque réseau minimale.

Pourquoi des contrôles en « rapport » sur les features ? Bloquer chaque feature sur des failles de Spring Boot 2.7.6, qui ne viennent pas du travail des membres, empêcherait toute l'équipe d'avancer. Les rapports sont publiés à chaque build pour que le problème reste visible. Le blocage s'applique sur `release` et `main`, là où se trouve le risque réel : la production.

12 Justification des choix technologiques

Besoin	Choix	Pourquoi	Alternative écartée
Branches	**GitFlow**	Features en parallèle, staging d'intégration, versions identifiées : adapté à une équipe de 5.	Trunk-based : exige des *feature flags*.
CI	**Jenkins Multibranch**	Pipeline versionné avec le code, un job par branche créé automatiquement. Standard en entreprise, exigé par le module.	GitHub Actions : moins de contrôle sur l'exécution.
Qualité	**SonarQube**	Porte qualité bloquante, historique par feature, tableau de bord unique.	Checkstyle ou PMD seuls : pas de porte qualité.
Sécurité	**Gitleaks, Trivy, ZAP**	Open source, rapides, complémentaires : secrets, dépendances et images, application en marche.	Snyk : payant. Dependency-Check : lent, sans images.
JAR	**Nexus**	Dépôt Maven privé, séparation *snapshots* / *releases*.	Publier uniquement dans Jenkins : pas de traçabilité.
Images	**Amazon ECR**	Accès par IAM (sans secret), tags immuables, scan intégré, natif avec EKS.	Registre Docker de Nexus : secrets et HTTPS à gérer.
Orchestration	**Kubernetes (EKS)**	Un namespace par feature, rolling update, autoscaling, auto-réparation. Plan de contrôle géré par AWS.	Docker Compose : pas d'isolation ni de mise à l'échelle.
Déploiement	**Argo CD (GitOps)**	Modèle *pull* : Jenkins sans droits sur le cluster. Correction automatique des dérives, retour arrière par Git, environnements créés automatiquement.	`kubectl` lancé depuis Jenkins : droits étendus, aucune traçabilité.

Besoin	Choix	Pourquoi	Alternative écartée
Paquetage K8s	**Helm**	Un seul modèle et trois profils (review, staging, prod) : pas de duplication.	Fichiers YAML copiés par environnement.
Provisionnement	**Terraform**	Infrastructure décrite et relue comme du code, `plan` avant d'agir, destruction et recréation à l'identique.	Console AWS : ni reproductible ni versionnée.
Configuration	**Ansible**	Sans agent (simple SSH), idempotent, secrets chiffrés avec Vault. Jenkins entièrement configuré par du code.	Scripts shell : ni idempotents ni lisibles.
Supervision	**Prometheus, Alertmanager, Grafana**	Standard du monde Kubernetes, découverte automatique, alertes fines.	CloudWatch : coûteux en métriques personnalisées.
Notification	**Slack**	Temps réel, un canal par source, intégrations natives avec les trois outils.	E-mail seul : peu réactif.

13 Mise en œuvre : exécuter le pipeline sur sa feature

Une fois l'infrastructure en place (section 6.4), chaque membre suit la même routine. Exemple avec Ahlem et la feature `logistics` :

#	Action	Ce qui se passe
1	Partir de `develop` à jour et créer la branche `feature/logistics`	La branche contient le Jenkinsfile commun à toute l'équipe.
2	Développer et lancer les tests en local (`mvn clean test`)	On détecte les erreurs avant même de pousser.
3	`git push`	Jenkins lance automatiquement « #1 · logistics ». Au premier build, Argo CD crée l'environnement `events-logistics`.
4	Suivre le résultat	Jenkins (étapes), SonarQube (`fst-events-logistics`), Argo CD (`events-logistics`), l'URL `logistics.<ip>.sslip.io/events/` , Grafana (Feature = `events-logistics`), Slack.
5	Ouvrir une Pull Request vers `develop`	Le pipeline vérifie la PR (étapes 0 à 6). Un coéquipier relit, puis on fusionne.
6	Supprimer l'environnement de la feature	Retirer le dossier `gitops/envs/logistics` du dépôt plateforme. Argo CD supprime tout seul l'application et ses ressources.
7	Fusion dans `develop`	Le pipeline de `develop` met à jour `events-staging`, où toutes les features sont intégrées et testées ensemble.

Où trouver quoi

Outil	Adresse	On y voit
Jenkins	`https://jenkins.<ip>.sslip.io`	Les builds de chaque branche, étape par étape, et les rapports.
SonarQube	`https://sonar.<ip>.sslip.io`	La qualité de chaque feature et son historique.
Nexus	`https://nexus.<ip>.sslip.io`	Les JAR publiés.
Argo CD	tunnel `kubectl port-forward`	L'état de chaque environnement, l'historique des déploiements.

Outil	Adresse	On y voit
Application	`http://<feature>.<ip>.sslip.io/events/`	Le dashboard et l'API de la feature.
Grafana	`http://grafana.<ip>.sslip.io`	Le dashboard « Events FST · par feature ».

Maîtriser le coût AWS

Quand tout tourne (EC2, plan de contrôle EKS, 2 nœuds, NAT, load balancers), l'ordre de grandeur est d'environ 0,5 $ par heure, à vérifier sur la calculatrice AWS. Entre deux séances, `terraform destroy` supprime tout. Le lendemain, `terraform apply` puis Ansible recréent la même plateforme. C'est l'un des bénéfices directs de l'Infrastructure as Code.

14 Limites actuelles et propositions d'amélioration

14.1 Limites assumées

Limite	Conséquence
Spring Boot 2.7.6, hors support	Failles connues dans les dépendances : les contrôles Trivy restent en rapport sur les features.
Schéma de base géré par `ddl-auto=update`	Pas de migrations versionnées : risqué en production.
MySQL dans un pod	Pas de sauvegarde automatique ni de haute disponibilité de la base.
Pas de NetworkPolicies	Les pods de namespaces différents peuvent communiquer entre eux.
Un seul serveur Jenkins avec un Docker partagé	Les builds des 5 membres se partagent les ressources de la même machine.
Une seule passerelle NAT	Point unique de défaillance pour la sortie Internet du cluster (choix d'économie).
Environnements de feature supprimés à la main	Risque d'oublier un environnement qui continue de coûter.

14.2 Propositions, par priorité

#	Amélioration	Ce que cela apporte	Effort
Priorité 1 : corriger les risques actuels			
1	**Spring Boot 3.x et Java 21**	Supprime les failles connues. Les contrôles Trivy peuvent devenir bloquants partout.	MOYEN
2	**Flyway**	Migrations SQL versionnées et rejouables à la place de `ddl-auto=update`.	FAIBLE
3	**Nettoyage automatique des environnements**	Quand une branche est supprimée, son environnement l'est aussi automatiquement.	FAIBLE
4	**NetworkPolicies « default deny »**	Seuls les flux prévus sont autorisés : ingress → application, application → MySQL, Prometheus → application.	FAIBLE
5	**External Secrets + AWS Secrets Manager**	Secrets centralisés avec rotation automatique.	MOYEN
Priorité 2 : fiabiliser les livraisons			
6	**Argo Rollouts (canary)**	La nouvelle version reçoit 10 %, puis 50 %, puis 100 % du trafic. Si Prometheus mesure une hausse des erreurs, **retour arrière automatique**.	MOYEN

#	Amélioration	Ce que cela apporte	Effort
7	**Tests de performance k6**	Une étape vérifie que la latence reste sous un seuil défini.	FAIBLE
8	**Amazon RDS MySQL** en production	Sauvegardes, haute disponibilité et mises à jour gérées par AWS.	MOYEN
9	**Agents Jenkins éphémères sur Kubernetes**	Chaque build dans un pod jetable : builds parallèles et isolés.	MOYEN
Priorité 3 : sécurité de la chaîne logicielle			
10	**SBOM (Syft) + signature Cosign**	Inventaire des composants de chaque image et preuve qu'elle vient bien de notre pipeline.	MOYEN
11	**Kyverno**	Le cluster refuse les images non signées, les pods root et les registres non autorisés.	MOYEN
12	**Pipeline pour l'infrastructure** (Checkov, `terraform plan` en PR)	Le code Terraform est analysé et relu avant d'être appliqué, et les écarts sont détectés chaque nuit.	MOYEN
13	**Renovate**	Pull Requests automatiques de mise à jour des dépendances.	FAIBLE
Priorité 4 : observabilité avancée et coûts			
14	**Loki + Tempo (OpenTelemetry)**	Journaux et traces dans Grafana, à côté des métriques.	MOYEN
15	**SLO et métriques DORA**	Objectifs de disponibilité mesurables, et suivi de la fréquence de déploiement, du délai de livraison, du taux d'échec et du temps de rétablissement.	MOYEN
16	**Karpenter + instances Spot, AWS Budgets**	Nœuds jusqu'à environ 70 % moins chers, environnements éteints la nuit, alerte de budget.	MOYEN

15 Conclusion

Ce projet met en place une chaîne DevOps complète pour l'application Events, construite autour d'une idée simple : **un seul pipeline, qui s'adapte à la branche qui l'exécute**. Chacun des cinq membres pousse sur sa feature et obtient automatiquement :

- son build, ses tests, son analyse de qualité ;
- ses rapports de sécurité, son image ;
- son environnement en ligne et sa supervision.

Toute l'équipe passe par exactement les mêmes contrôles. L'architecture sépare clairement les responsabilités :

Jenkins
Construit, teste, contrôle et publie.

Argo CD
Déploie à partir de Git (GitOps), sans que Jenkins ait accès au cluster.

Prometheus, Alertmanager, Grafana
Mesurent, alertent et montrent.

Terraform et Ansible
Décrivent toute l'infrastructure AWS en code, reproductible et destructible à volonté.

La sécurité est intégrée à chaque niveau : secrets, code, dépendances, image, application en marche, cluster et cloud. Les limites actuelles sont identifiées, et la section 14 propose une feuille de route pour aller plus loin : montée de version de Spring Boot, politiques réseau, déploiements progressifs, signature des images et politiques de sécurité dans le cluster.

En une phrase

Un `git push` sur une feature déclenche 17 étapes automatisées, réalisées par plus de 20 technologies, et se termine par une application en ligne, vérifiée, sécurisée et supervisée.

A Glossaire

Artefact	Résultat livrable d'un build : ici le JAR (dans Nexus) et l'image Docker (dans ECR).
CI / CD	Intégration continue : construire et tester à chaque modification. Livraison et déploiement continus : publier et mettre en ligne automatiquement.
CVE	Identifiant public d'une faille de sécurité connue (ex. CVE-2022-1471).
DAST	Test de sécurité dynamique : analyser l'application en fonctionnement (OWASP ZAP).
DevSecOps	Intégrer la sécurité à chaque étape de la chaîne DevOps, et non à la fin.
GitFlow	Modèle de branches : `feature/*`, `develop`, `release/*`, `main`, `hotfix/*`.
GitOps	Git décrit l'état voulu du système, et un agent (Argo CD) l'applique et le maintient en continu.
Helm (chart)	Modèle paramétrable de ressources Kubernetes, réutilisé pour tous les environnements.
HPA	Mise à l'échelle automatique : Kubernetes ajoute des pods quand la charge CPU augmente.
IaC	Infrastructure as Code : l'infrastructure est décrite dans des fichiers versionnés (Terraform, Ansible).
Idempotence	Relancer une opération donne le même résultat, sans effet de bord (propriété d'Ansible et de Terraform).
IAM (rôle)	Identité AWS attachée à une machine, qui lui donne des droits sans clé stockée.
Namespace	Espace isolé dans Kubernetes : un par feature dans notre projet.
NetworkPolicy	Règle Kubernetes qui autorise ou refuse le trafic entre pods (amélioration proposée).
Quality Gate	Seuils de qualité SonarQube qui décident si le code peut être livré.
Rolling update	Remplacement progressif des pods, sans interruption de service.
SAST / SCA	Analyse statique du code (SonarQube) et analyse des composants tiers (Trivy).
ServiceMonitor	Ressource qui indique à Prometheus quelle application surveiller, et où.
Smoke test	Vérification rapide qu'une application déployée répond et fonctionne.
Synced / Healthy	États Argo CD : le cluster est conforme à Git, et l'application fonctionne.